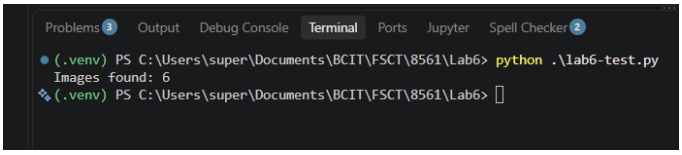
Braeden Venne

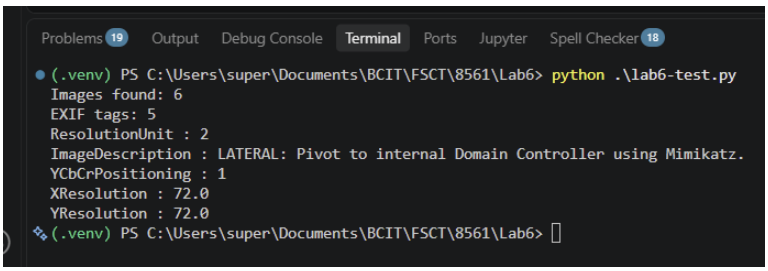
Lab 6: Covert Channels in Image Metadata

Part 0 – Prepare the environment

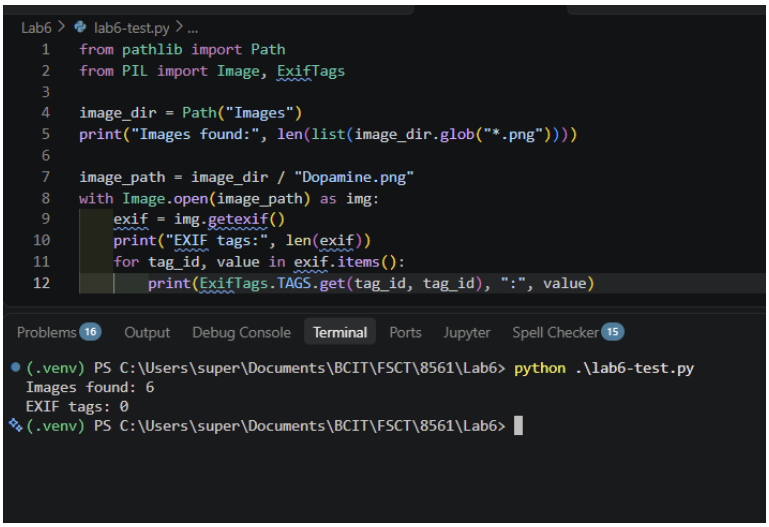


- Prepared and tested the environment

Part 1 - First technique: EXIF metadata (one image)

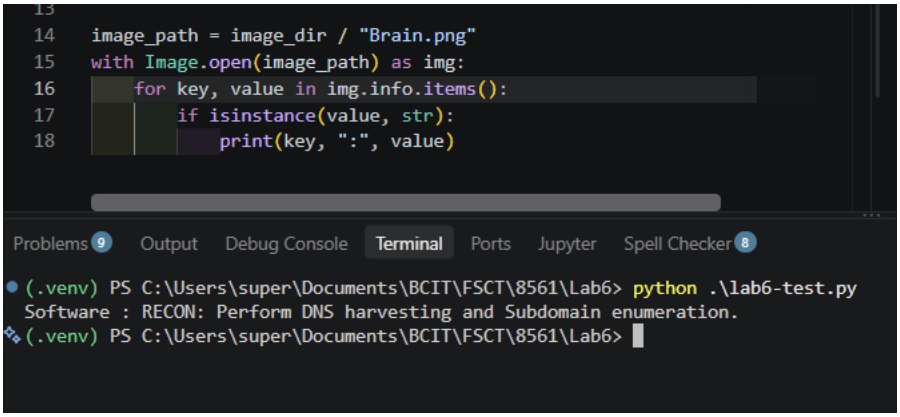


- Which EXIF field carries a message? Record it in your evidence log.
    - ImageDescription carries a message “LATERAL: Pivot to internal Domain Controller using Mimikatz.”

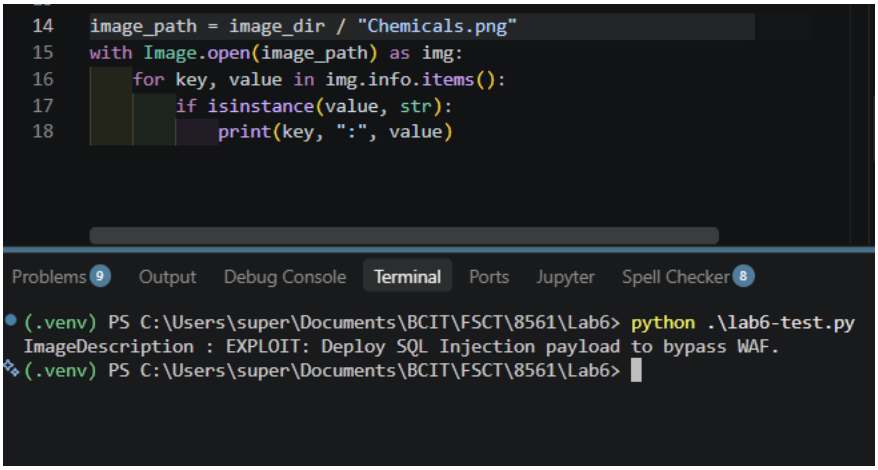


- I tried changing the image_path variable to “Dopamine.png” and it returned “EXIF tags: 0”. Having EXIF tags be 0 does not indicate that there is no metadata, EXIF is just one of the many metadata categories that contains info such as the camera model, shutter speed, ISO, focal length, and GPS.

Part 2 - Second technique: PNG text metadata (one image)



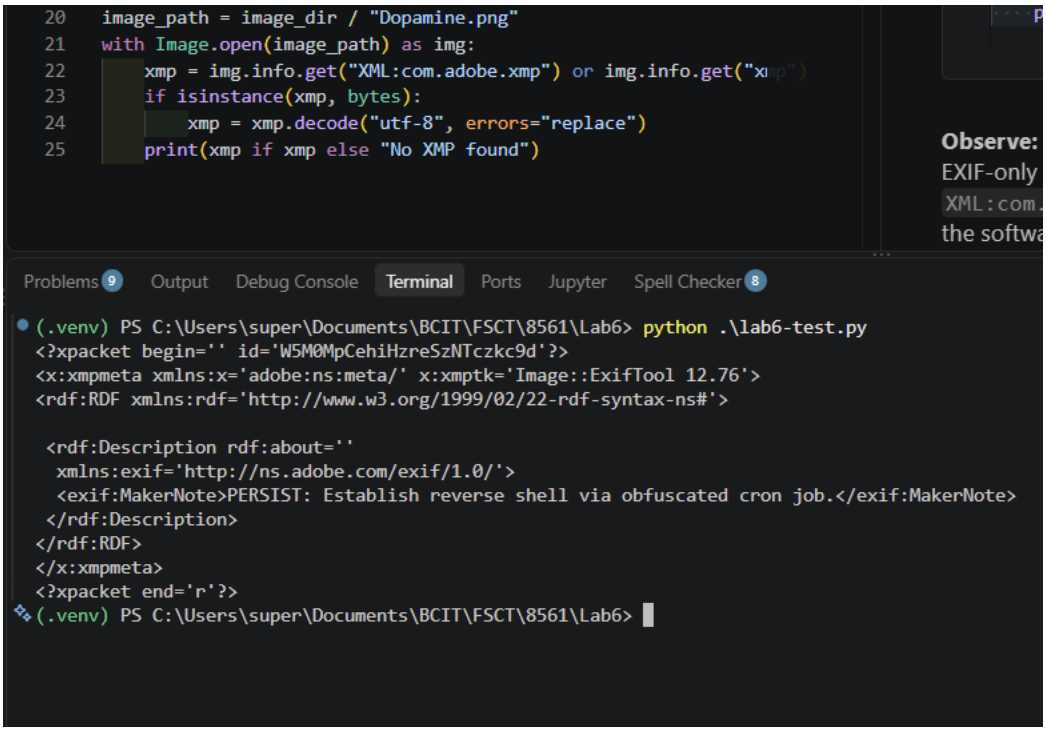
- **Which PNG text field contains a clue? Try Chemicals.png next using the same short snippet.**
    - In Brain.png, the Software PNG text field contains the clue “RECON: Perform DNS harvesting and Subdomain enumeration”.



- In Chemincals.png, the ImageDescription PNG text field contains the clue “EXPLOIT: Deploy SQL Injection payload to bypass WAF”.

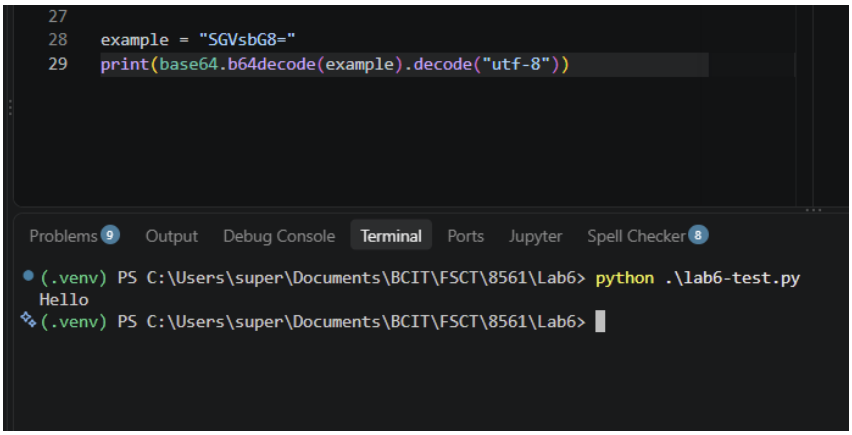
This demonstrates that PNG files can store valuable metadata in various text chunks, such as Software or ImageDescription, that are distinct from the standard EXIF fields and don’t appear when using getexif().

Part 3 - Third technique: XMP metadata (one image)

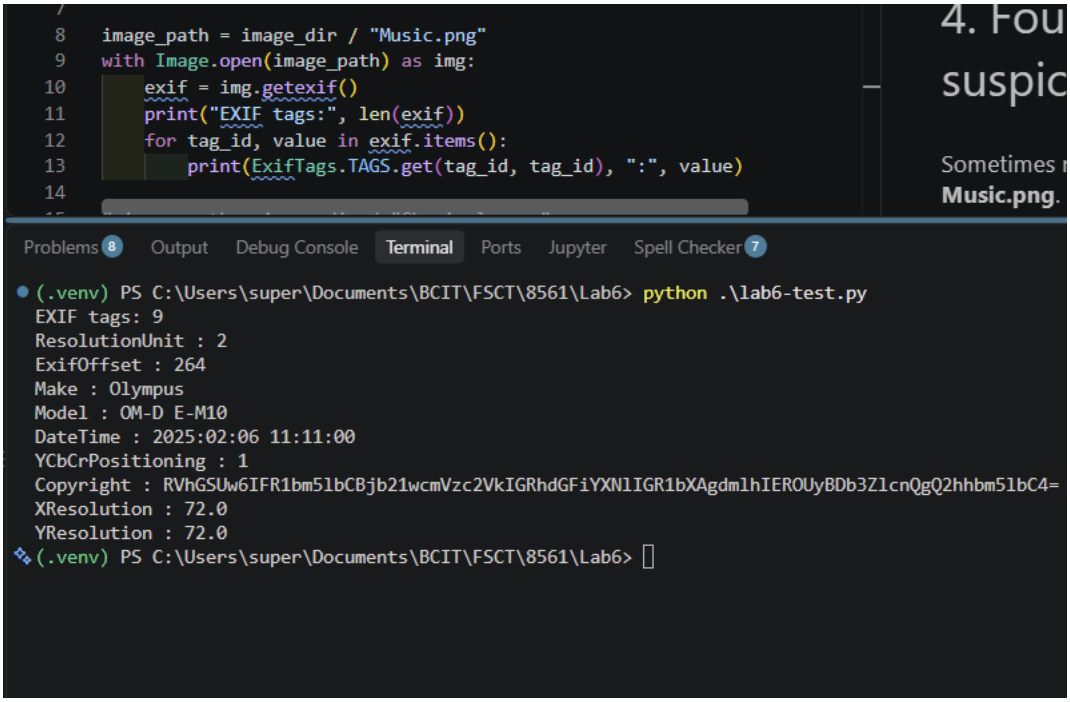


- Locate the XML element containing the partial secret. Why did the EXIF-only approach miss it? Note: XMP may appear under either `XML:com.adobe.xmp` or `xmp`, depending on how the file was created and the software version.
    - Inspecting Dopamine.png using the XMP extraction snippet uncovered the following XML metadata block containing the partial secret:
        - <exif:MakerNote>PERSIST: Establish reverse shell via obfuscated cron job.</exif:MakerNote>
    - The EXIF-only approach missed it because getexif() only looks at standard EXIF tags, XMP data is stored differently, it uses XML packets embedded within separate parts of the file structure, making EXIF parsers completely skip over it unless it specifically checks for XMP.

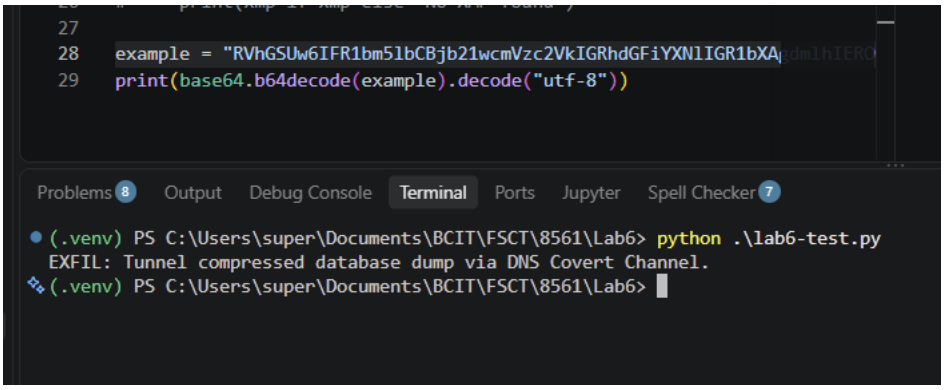
Part 4 - Fourth technique: Decode suspicious text (one image)



- Replace `example` with the suspicious metadata value from Music.png. Do not assume every long string is Base64; use the context and validate the result.

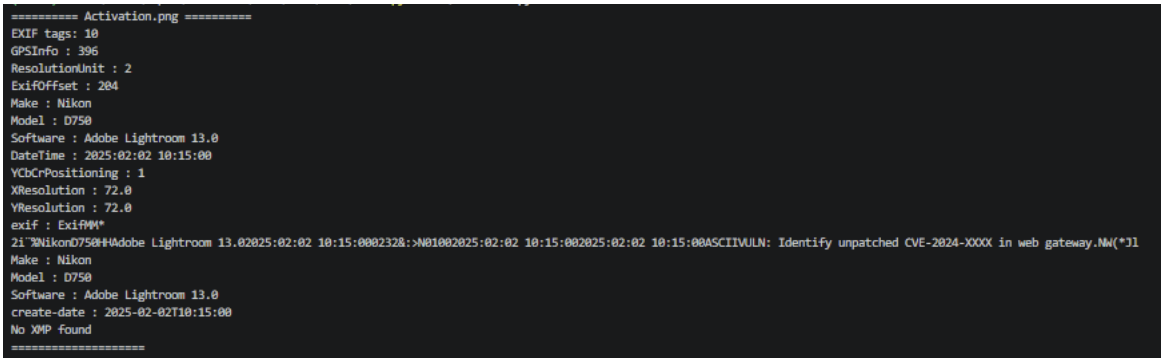


- First, I checked the EXIF and PNG text fields, it appears that the “Copyright” field is in base64.

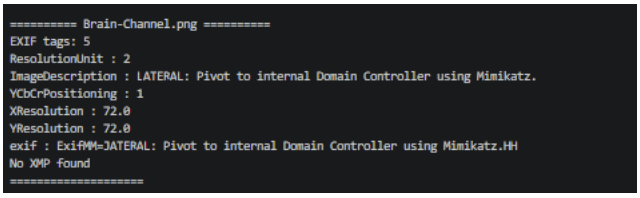


- I then took the “Copyright” value and placed it in the “example” variable to devcode the message and it returned “EXFIL: Tunnel compressed database dump via DNS Covert Channel.”

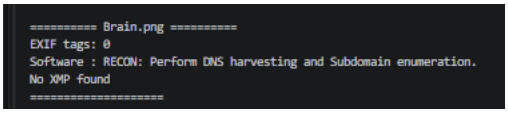
Part 5 - Your investigation: six images, one by one



- Results for Activation.png

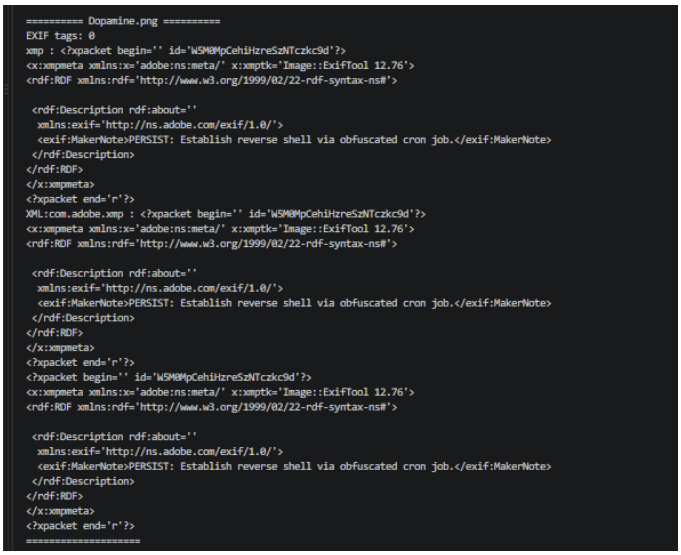


- Results for Brain-Channel.png



- Results for Brain.png

Braeden Venne



- Results for Dopamine.png



- Results for Music.png

Image	Metadata type / field	Decoding needed?	Partial secret recovered
Activation.png	EXIF	No	VULN: Identify unpatched CVE-2024-XXXX in web gateway
Brain-Channel.png	EXIF ImageDescription	No	LATERAL: Pivot to internal Domain Controller using Mimikatz.
Brain.png	PNG Info Software	No	RECON: Perform DNS harvesting and Subdomain enumeration.
Chemicals.png	PNG Info ImageDescription	No	EXPLOIT: Deploy SQL Injection payload to bypass WAF.

Dopamine.png	XMP MakerNote	No	PERSIST: Establish reverse shell via obfuscated cron job.
Music.png	PNG Info Copyright	Yes	EXFIL: Tunnel compressed database dump via DNS Covert Channel

Part 6 - Reconstruct the complete secret

1. Stage 1: RECON - Recovered Message: Perform DNS harvesting and Subdomain enumeration.

2. Stage 2: VULN - Recovered Message: Identify unpatched CVE-2024-XXXX in web gateway

3. Stage 3: EXPLOIT - Recovered Message: Deploy SQL Injection payload to bypass WAF.

4. Stage 4: LATERAL - Recovered Message: Pivot to internal Domain Controller using Mimikatz.

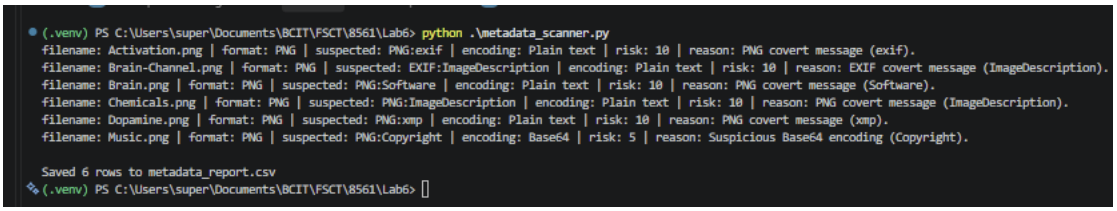
5. Stage 5: PERSIST - Recovered Message: Establish reverse shell via obfuscated cron job.

6. Stage 6: EXFIL - Recovered Message: Tunnel compressed database dump via DNS Covert Channel

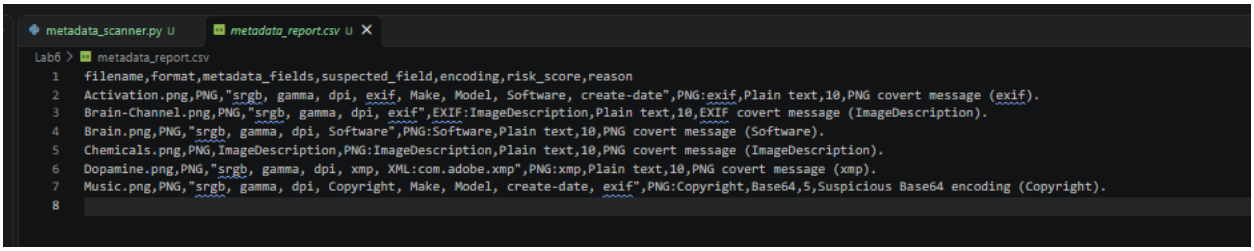
Reasoning of order

This sequence follows the standard for a cyberattack. Reconnaissance occurs first to gather information on the target. Vulnerability analysis identifies the specific weakness to the target. Exploitation breaches identified weaknesses to gain access. Lateral movement expands the attackers access to internal systems. Persistence ensures long-term access. Lastly, exfiltration is the final objective of the attacker where data is stolen out of the environment.

Part 7 - Build an automated metadata scanner



- Terminal output for “metadata_scanner.py” showing the findings in the provided pictures.



- CSV file “metadata_report.csv” containing all the findings

Discussion:

A major limitation to EXIF-only extraction is that relying on only EXIF data completely ignores other places where important metadata can hide, such as PNG text chunks or XMP packets. Since malware or covert messages are often stored outside of standard camera tags, an EXIF-only scanner would completely miss crucial evidence. Additionally, automated risk scoring also comes with its own limitations as they heavily rely on static keyword lists. With automated risk scoring you can easily flag safe files as dangerous just because they contain a common security term, or miss a hidden threat that doesn’t match the exact pattern built into the script.

Part 8 - Reflection Questions

1. **Why can an image contain a hidden message even when getexif() returns no tags?**
    a. The getexif() function only extracts standard Exchangeable Image File Format (EXIF) tags, which typically only store hardware-specific metadata such as camera models, exposure settings, and timestamps. However, image file formats such as PNG store metadata in structures such as generic text chunks or Extensible Metadata Platform (XMP) XML packets, resulting to an image returning zero EXIF tags while still containing valuable information like hidden messages.

2. **How is encoding different from encryption? Is Base64 a secure way to hide information?**
    a. Encoding converts data into a standardized format using a publicly known scheme, such as Base64 or UTF-8, to ensure it can be safely stored or transmitted across various systems, anyone who recognizes the scheme can easily reverse with no issues. Encryption uses a secret key and mathematical algorithm to scramble data, making the content unreadable without the corresponding key. Base64 is not a secure method for hiding information because it provides no encryption or access protection, making it easy to decode.

3. **Why should investigators preserve the original files and record hashes?**
    a. Investigators should preserve the original files to maintain the chain of custody, ensuring evidence is not altered or corrupted during analysis. Additionally, they must also record hashes as it establishes a unique digital signature for each file,

allowing investigators to verify file integrity and prove in court or an audit that the evidence has not been altered.

4. **Can metadata alone prove that an image was edited or that an incident occurred? What corroborating evidence would you need?**
    a. Metadata alone cannot prove that an image was edited or that an incident occurred because metadata fields can easily be spoofed, stripped, modified, or forged using modern tools. Investigators need corroborating evidence such as system event logs, application logs, file system timeline analysis, or cryptographic file integrity checks to verify that an actual incident took place.

5. **How does this investigation connect to the automated evidence collection and analysis you practised in Labs 4 and 5?**
    a. This lab builds directly on the automated parsing and scanning techniques introduced in Labs 4 and 5. While Lab 4 automated network traffic parsing with Scapy and Lab 5 used the requests library to query web endpoints and analyze server responses, Lab 6 translates those same core scripting principles, such as structured looping, checking for specific conditions, and building structured reports, to file metadata. Overall, combining these labs creates some powerful automated evidence and analysis tools that each cover different aspects of a system, making them useful cybersecurity/forensic tools.